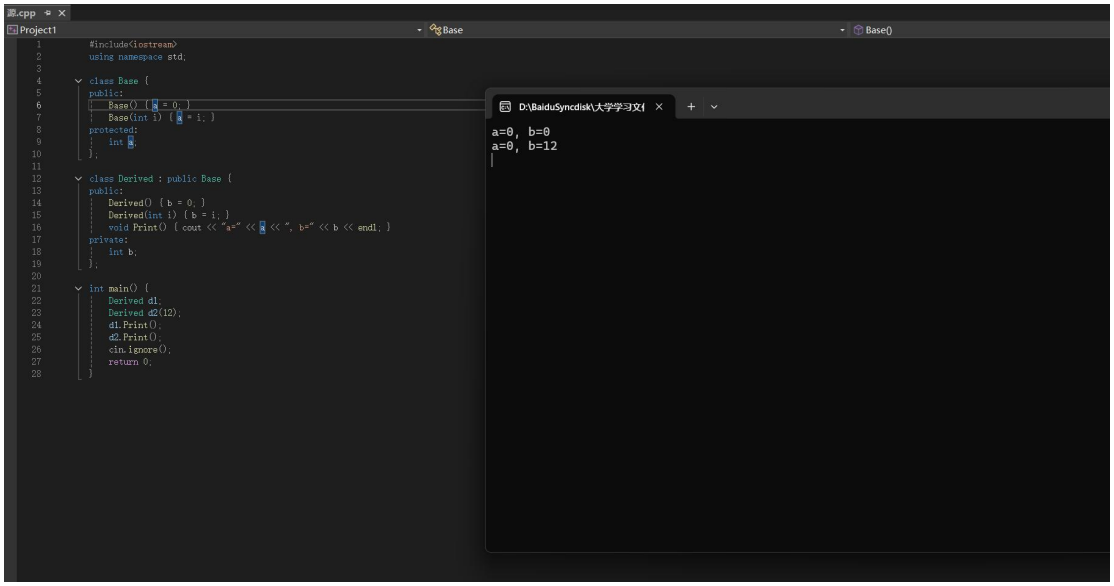
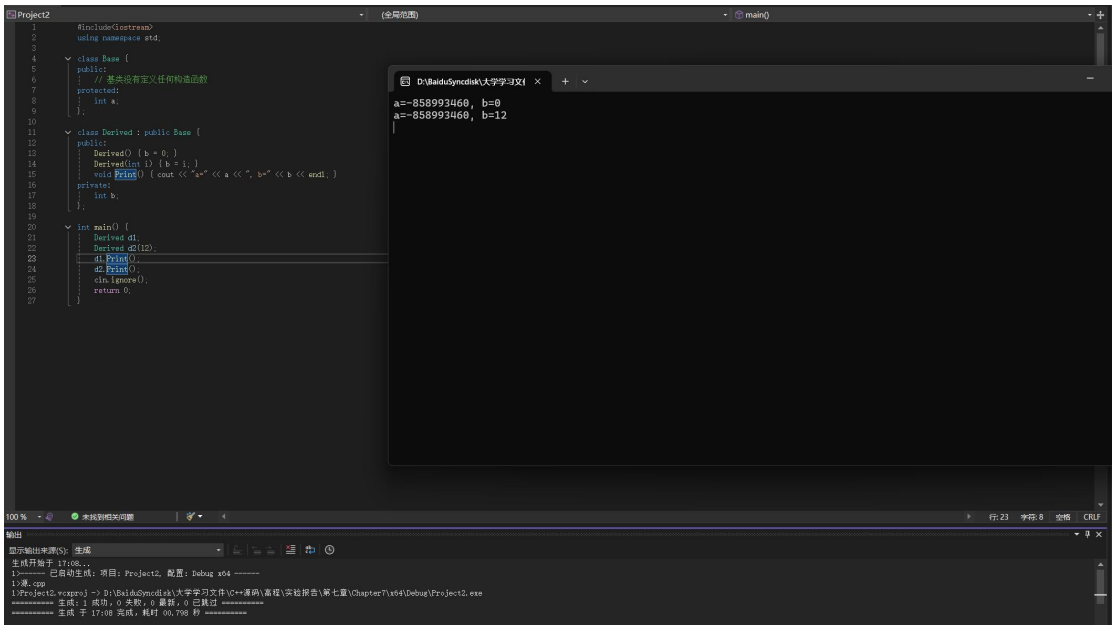
第七章---类的继承

练习 1.



练习 2



练习 3

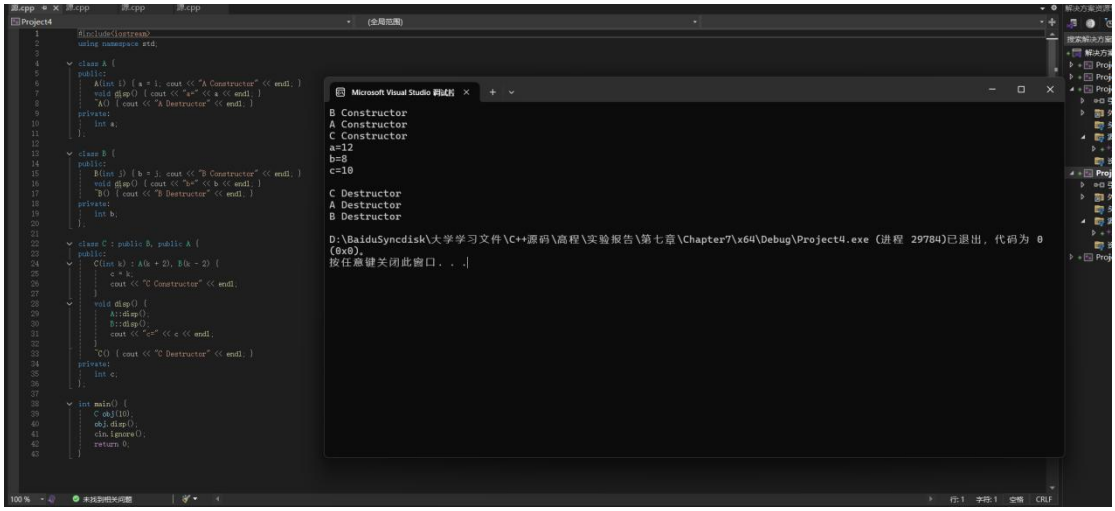
```cpp
#include<iostream>
using namespace std;

class Base {
public:
    //Base() { a = 0; }  // 注释掉了默认构造函数
    Base(int i) { a = i; }
protected:
    int a;
};

class Derived : public Base {
public:
    // 这里会出错，因为Derived()试图调用Base()，但Base()不存在
    Derived() { b = 0; }  // 错误：没有合适的默认构造函数可用
    Derived(int i) { b = i; }  // 错误：需要显式调用基类构造函数
    void Print() { cout << "a=" << a << ", b=" << b << endl; }
private:
    int b;
};

// 修正版本：
class DerivedFixed : public Base {
public:
    DerivedFixed() : Base(0) { b = 0; }  // 正确：显式调用基类构造函数
    DerivedFixed(int i) : Base(i) { b = i; }  // 正确：显式调用基类构造函数
    void Print() { cout << "a=" << a << ", b=" << b << endl; }
private:
    int b;
};

int main() {
    // Derived d1;  // 这行会编译错误
    // Derived d2(12);  // 这行也会编译错误

    DerivedFixed d1;
    DerivedFixed d2(12);
    d1.Print();
    d2.Print();
    cin.ignore();
    return 0;
}
```

Derived()试图调用 Base()，但 Base()不存在

```cpp
#include<iostream>
using namespace std;

class Base {
public:
    //Base() { a = 0; }  // 注释掉了默认构造函数
    Base(int i) { a = i; }
protected:
    int a;
};

/*
class Derived : public Base {
public:
    // 这里会出错，因为Derived()试图调用Base()，但Base()不存在
    Derived() { b = 0; }  // 错误：没有合适的默认构造函数可用
    Derived(int i) { b = i; }  // 错误：需要显式调用基类构造函数
    void Print() { cout << "a=" << a << ", b=" << b << endl; }
private:
    int b;
};
*/

// 修正版本：
class DerivedFixed : public Base {
public:
    DerivedFixed() : Base(0) { b = 0; }  // 正确：显式调用基类构造函数
    DerivedFixed(int i) : Base(i) { b = i; }  // 正确：显式调用基类构造函数
    void Print() { cout << "a=" << a << ", b=" << b << endl; }
private:
    int b;
};

int main() {
    // Derived d1;  // 这行会编译错误
    // Derived d2(12);  // 这行也会编译错误

    DerivedFixed d1;
    DerivedFixed d2(12);
    d1.Print();
    d2.Print();
    cin.ignore();
    return 0;
}
```

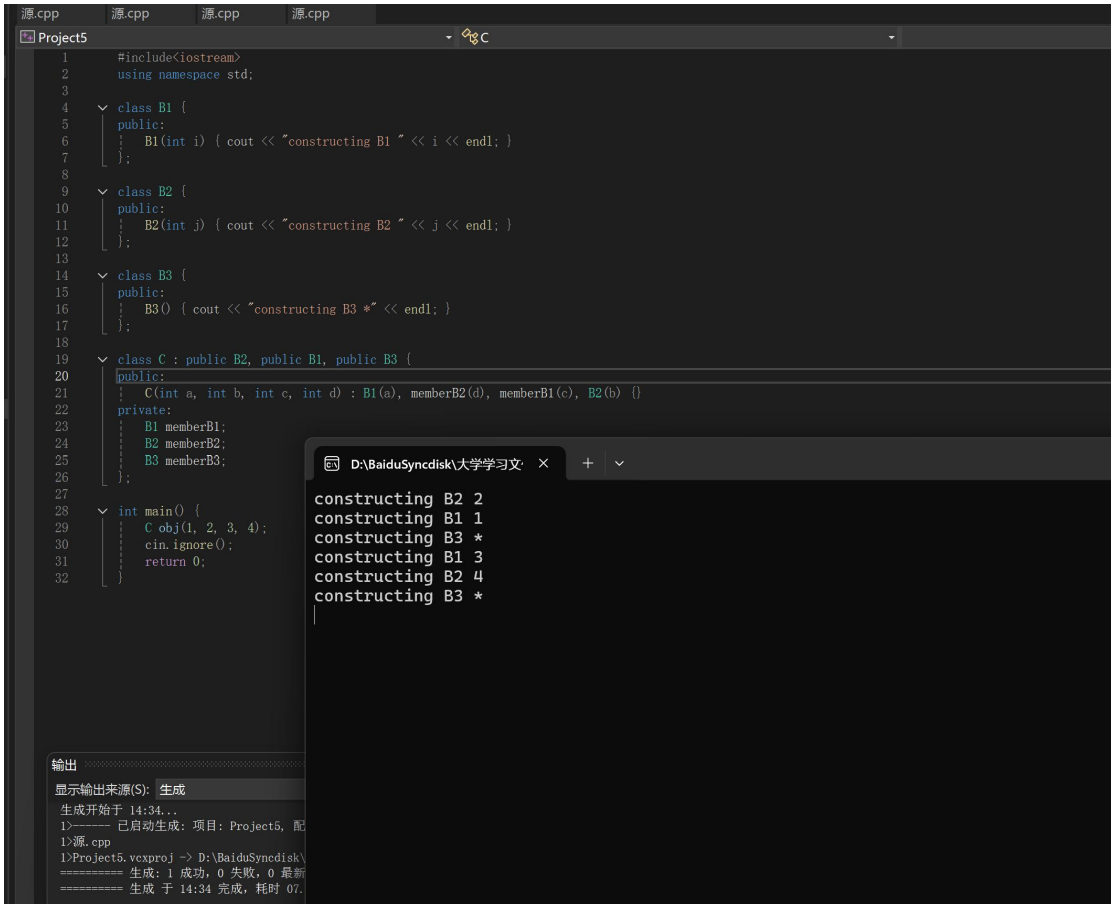
```
a=0, b=0
a=12, b=12

D:\BaiduSyncdisk\大学学习文件\C++源码\高程\实验报告\第七章\Chapter7\x64\Debug\Project3.exe (进程 15948)已退出，
(0x0)。
按任意键关闭此窗口. . .
```

练习 4



构造函数顺序为 B,A,C；析构函数顺序为 C,A,B

练习 5

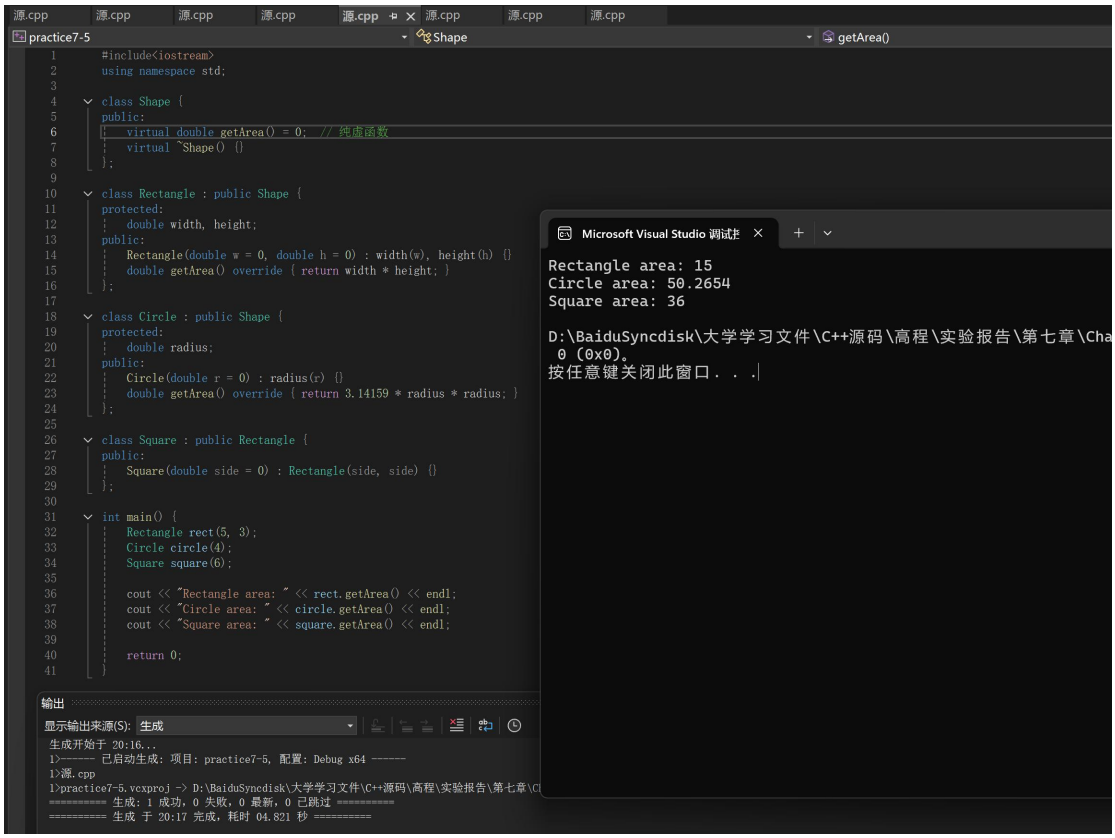


1.基类构造顺序　（由继承顺序决定）：
B2(b)→ "constructing B2 2"　（参数 b=2）
B1(a)→ "constructing B1 1"　（参数 a=1）
B3()→ "constructing B3 *"　（无参数）

2.成员对象构造顺序　（由声明顺序决定）：
B1 memberB1(c)→ "constructing B1 3"　（参数 c=3）
B2 memberB2(d)（参数 d=4）
B3 memberB3()

基类构造顺序　：由 class C : public B2, public B1, public B3 的声明顺序决定
成员对象构造顺序　：由类中成员变量的声明顺序决定
初始化列表顺序不影响实际构造顺序

习题 7-5



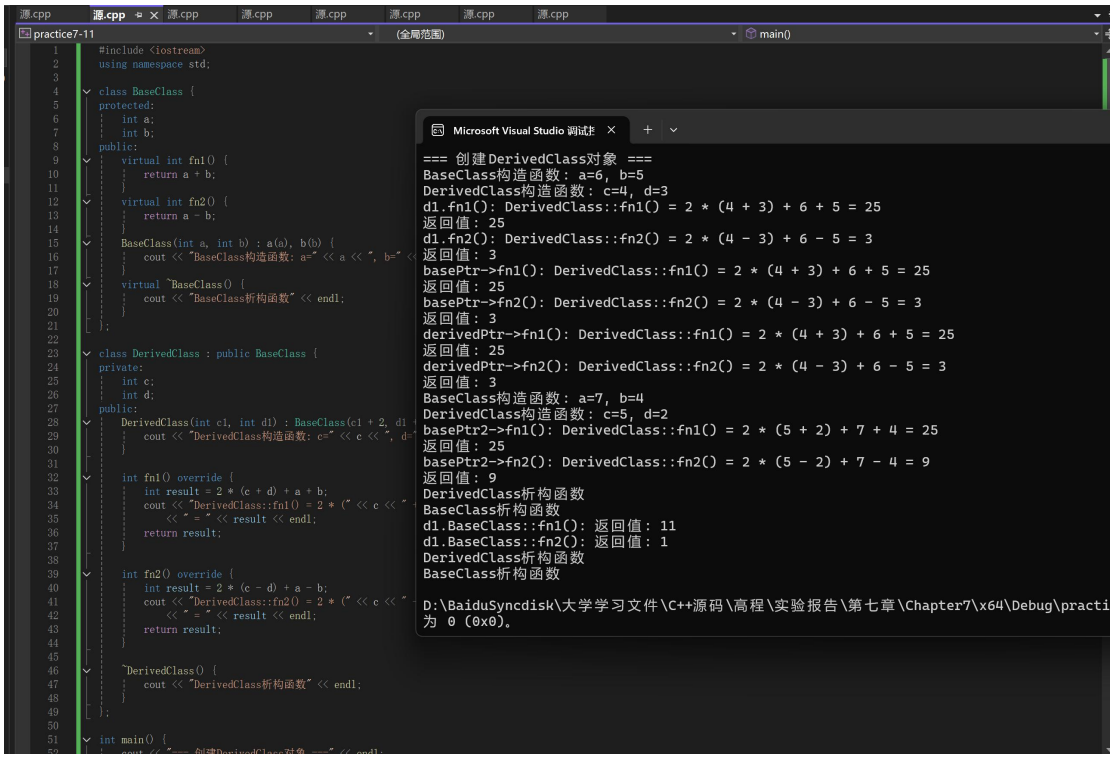
（1）virtual：声明为虚函数，允许子类重写
（2）= 0：表示这是纯虚函数，使 Shape 成为抽象类
（3）作用：定义了一个接口契约，所有继承自 Shape 的类都必须实现自己的面积计算方法
（4）override：明确表示重写父类的虚函数

1. 虚函数表（vtable）

cpp　　复制　下载

```cpp
// 创建Shape指针数组展示多态
Shape* shapes[] = {&rect, &circle, &square};

for (Shape* shape : shapes) {
    cout << "Area: " << shape->getArea() << endl;
    // 虽然都是Shape指针，但会调用各自实际的getArea()
}
```

习题 7-11



（1）多态性：通过基类指针调用虚函数时，实际调用的是派生类重写的版本
（2）动态绑定：使用 new 创建的派生类对象赋值给基类指针，展示了真正的运行时多态
（3）作用域解析：d1.BaseClass::fn1()强制调用基类版本的函数，忽略派生类的重写
（4）虚函数表：C++通过虚函数表实现多态，确保正确调用重写的函数

习题 7-15

```cpp
#include<iostream>
using namespace std;

struct Base1 { int x; };
struct Base2 { float y; };
struct Derived : Base1, Base2 {};

int main() {
    Derived* pd = new Derived;
    pd->x = 1;
    pd->y = 2.0f;

    void* pv = pd;
    Base2* pb = static_cast<Base2*>(pv);  // 这里有问题！

    cout << pd->y << " " << pb->y << endl;

    delete pd;  // 应该删除pd，而不是pb
    return 0;
}
```

```cpp
// 修正版本：
int main_fixed() {
    Derived* pd = new Derived;
    pd->x = 1;
    pd->y = 2.0f;

    // 正确的方式：使用dynamic_cast或直接转换
    Base2* pb = pd;  // 隐式转换，正确

    cout << pd->y << " " << pb->y << endl;

    delete pd;
    return 0;
}
```